1. Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

```
import java.io.*;
import java.util.*;

class Farmer {
    String name, fruit, city;
    int quantity;

    Farmer(String name, String fruit, String city, int quantity) {
        this.name = name;
        this.fruit = fruit;
        this.city = city;
        this.quantity = quantity;
    }
}

public class FarmerData {

    // Method to write data to file
    public static void writeToFile(String filename, Farmer[] farmers) {
        try {
            BufferedWriter bw = new BufferedWriter(new FileWriter(filename));

            bw.write("Name\tFruit\tCity\tQuantity\n");
            bw.write("----------------------------------------\n");

            for (Farmer f : farmers) {
                bw.write(f.name + "\t" + f.fruit + "\t" + f.city + "\t" +
f.quantity + "\n");
            }

            bw.close();
            System.out.println("Data written to " + filename);
        } catch (IOException e) {
            System.out.println("Error writing file");
        }
    }

    // Search function
    public static void searchFarmer(Farmer[] farmers, String searchName) {
```

```java
        boolean found = false;

        for (Farmer f : farmers) {
            if (f.name.equalsIgnoreCase(searchName)) {
                System.out.println("\nFarmer Found:");
                System.out.println("Name: " + f.name);
                System.out.println("Fruit: " + f.fruit);
                System.out.println("City: " + f.city);
                System.out.println("Quantity: " + f.quantity);
                found = true;
                break;
            }
        }

        if (!found) {
            System.out.println("Farmer not found!");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Farmer[] farmers = new Farmer[10];

        // Input data
        for (int i = 0; i < 10; i++) {
            System.out.println("\nEnter details of Farmer " + (i + 1));

            System.out.print("Name: ");
            String name = sc.nextLine();

            System.out.print("Fruit: ");
            String fruit = sc.nextLine();

            System.out.print("City: ");
            String city = sc.nextLine();

            System.out.print("Quantity: ");
            int quantity = sc.nextInt();
            sc.nextLine(); // consume newline

            farmers[i] = new Farmer(name, fruit, city, quantity);
        }

        // Save before sorting
        writeToFile("before_sort.txt", farmers);

        // Sorting based on quantity (ascending)
        Arrays.sort(farmers, Comparator.comparingInt(f -> f.quantity));
```

```java
        // Save after sorting
        writeToFile("after_sort.txt", farmers);

        // Search operation
        System.out.print("\nEnter farmer name to search: ");
        String searchName = sc.nextLine();

        searchFarmer(farmers, searchName);

        sc.close();
    }
}
```

OUTPUT :-

Enter details of Farmer 1

Name: Sanjay

Fruit: Banana

City: Kapartunga

Quantity: 70

Enter details of Farmer 2

Name: Anish

Fruit: Apple

City: Bilaspur

Quantity: 50

Enter details of Farmer 3

Name: Raunak

Fruit: Watermelon

City: Kawardha

Quantity: 67

Enter details of Farmer 4

Name: Aditya

Fruit: Mango

City: Sakti

Quantity: 69

Enter details of Farmer 5

Name: Mohit

Fruit: Litchi

City: Kharsia

Quantity: 80

Enter details of Farmer 6

Name: Ayush

Fruit: Orange

City: Dholakpur

Quantity: 20

Enter details of Farmer 7

Name: Pratik

Fruit: Grapes

City: Bilaspur

Quantity: 78

Enter details of Farmer 8

Name: Manas

Fruit: Papaya

City: Janjgir

Quantity: 2

Enter details of Farmer 9

Name: Babita

Fruit: Muskmelon

City: Mumbai

Quantity: 69

Enter details of Farmer 10

Name: Elon musk

Fruit: Muskmelon

City: Raipur

Quantity: 96

Data written to before_sort.txt

Data written to after_sort.txt

Enter farmer name to search: Manas

Farmer Found:

Name: Manas

Fruit: Papaya

City: Janjgir

Quantity: 2

2. Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

```java
import java.util.Scanner;

class Fruit {
    String name;
    int quantity;
    String quality;

    Fruit(String name, int quantity, String quality) {
        this.name = name;

        // If quantity > 1000, store half
        if (quantity > 1000) {
            this.quantity = quantity / 2;
        } else {
            this.quantity = quantity;
        }

        this.quality = quality.toLowerCase();
    }

    void display() {
        System.out.println(name + " | Quantity: " + quantity + " | Quality: " + quality);
    }
}

public class Fruit_management {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Fruit[] fruits = new Fruit[3];
        String names[] = {"Mango", "Apple", "Papaya"};

        // Input
        for (int i = 0; i < 3; i++) {
            System.out.print("Enter quantity of " + names[i] + ": ");
            int qty = sc.nextInt();

            System.out.print("Enter quality (good/bad/average): ");
            String quality = sc.next();

            fruits[i] = new Fruit(names[i], qty, quality);
        }
```

```java

        // Display stored data
        System.out.println("\nStored Data:");
        for (Fruit f : fruits) {
            f.display();
        }

        // Search
        System.out.print("\nEnter fruit name to search: ");
        String searchName = sc.next().toLowerCase();

        System.out.print("Enter quality to search: ");
        String searchQuality = sc.next().toLowerCase();

        boolean found = false;

        for (Fruit f : fruits) {
            if (f.name.toLowerCase().equals(searchName) && f.quality.equals(searchQuality)) {
                System.out.println("Fruit Found:");
                f.display();
                found = true;
            }
        }

        if (!found) {
            System.out.println("No matching fruit found.");
        }

        sc.close();
    }
}
```

OUTPUT :-

Enter quantity of Mango: 40

Enter quality (good/bad/average): good

Enter quantity of Apple: 69

Enter quality (good/bad/average): bad

Enter quantity of Papaya: 30

Enter quality (good/bad/average): average

Stored Data:

Mango | Quantity: 40 | Quality: good

Apple | Quantity: 69 | Quality: bad

Papaya | Quantity: 30 | Quality: average

Enter fruit name to search: Mango

Enter quality to search: good

Fruit Found:

Mango | Quantity: 40 | Quality: good

76. There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively. The available orchards in a tree-view. Search for a given fruit in the available orchards after taking input from the user & print the tree.

```
import java.util.*;

class Orchard {
    String name;
    String fruit;
    String quality;

    Orchard(String name, String fruit, String quality) {
        this.name = name;
        this.fruit = fruit;
        this.quality = quality;
    }
}

public class OrchardTree {

    // Display Tree
    public static void displayTree(Orchard[] orchards) {
        System.out.println("\nTree Structure:");
        System.out.println("           Orchards");

        for (Orchard o : orchards) {
            System.out.println("            |");
            System.out.println("           " + o.name);
            System.out.println("        (" + o.fruit + ", " + o.quality +
")");
        }
    }

    // Search fruit
    public static void searchFruit(Orchard[] orchards, String fruit) {
        boolean found = false;

        System.out.println("\nSearch Result for fruit: " + fruit);

        for (Orchard o : orchards) {
            if (o.fruit.equalsIgnoreCase(fruit)) {
```

```
                System.out.println("Found in Orchard " + o.name +
                                   " (Quality: " + o.quality + ")");
                found = true;
            }
        }

        if (!found) {
            System.out.println("Fruit not found in any orchard!");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Initialize orchards
        Orchard[] orchards = {
            new Orchard("A", "Apple", "Good"),
            new Orchard("B", "Litchi", "Good"),
            new Orchard("C", "Mango", "Good"),
            new Orchard("D", "Apple", "Bad")
        };

        // Display tree
        displayTree(orchards);

        // Take input
        System.out.print("\nEnter fruit to search: ");
        String fruit = sc.nextLine();

        // Search operation
        searchFruit(orchards, fruit);

        sc.close();
    }
}
```

OUTPUT :-

```
Tree Structure:
        Orchards
         |
        A
      (Apple, Good)
         |
        B
      (Litchi, Good)
         |
        C
      (Mango, Good)
         |
        D
      (Apple, Bad)

Enter fruit to search: Mango

Search Result for fruit: Mango
Found in Orchard C (Quality: Good)
```

4. Take a map provided as a graph & display all the subtrees of connected points.

```
import java.util.*;

public class DFS {

    // DFS to find connected component
    static void dfs(int node, boolean[] visited, List<List<Integer>> graph,
List<Integer> component) {
        visited[node] = true;
        component.add(node);

        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                dfs(neighbor, visited, graph, component);
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of nodes: ");
        int n = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        // Adjacency list
        List<List<Integer>> graph = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        // Input edges
        System.out.println("Enter edges (u v):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();

            graph.get(u).add(v);
            graph.get(v).add(u); // undirected graph
        }

        boolean[] visited = new boolean[n];
```

```java
        System.out.println("\nSubtrees (Connected Components):");

        // Find all components
        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                List<Integer> component = new ArrayList<>();
                dfs(i, visited, graph, component);

                // Print component
                System.out.println(component);
            }
        }

        sc.close();
    }
}
```

OUTPUT :-

Enter number of nodes: 5

Enter number of edges: 3

Enter edges (u v):

0 1

1 2

3 4

Subtrees (Connected Components):

[0, 1, 2]

[3, 4]

5. Enter entry point & goal point & produce the matching subtree of the graph.

```java
import java.util.*;

public class Graph {

    // DFS to find path
    static boolean dfs(int current, int goal, boolean[] visited,
                       List<List<Integer>> graph, List<Integer> path) {

        visited[current] = true;
        path.add(current);

        if (current == goal) {
            return true;
        }

        for (int neighbor : graph.get(current)) {
            if (!visited[neighbor]) {
                if (dfs(neighbor, goal, visited, graph, path)) {
                    return true;
                }
            }
        }

        // backtrack if path not found
        path.remove(path.size() - 1);
        return false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of nodes: ");
        int n = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        List<List<Integer>> graph = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        System.out.println("Enter edges (u v):");
        for (int i = 0; i < e; i++) {
```

```java
            int u = sc.nextInt();
            int v = sc.nextInt();

            graph.get(u).add(v);
            graph.get(v).add(u);
        }

        System.out.print("Enter entry point: ");
        int start = sc.nextInt();

        System.out.print("Enter goal point: ");
        int goal = sc.nextInt();

        boolean[] visited = new boolean[n];
        List<Integer> path = new ArrayList<>();

        if (dfs(start, goal, visited, graph, path)) {
            System.out.println("Matching Subtree (Path): " + path);
        } else {
            System.out.println("No path found.");
        }

        sc.close();
    }
}
```

OUTPUT :-

Enter number of nodes: 5

Enter number of edges: 4

Enter edges (u v):

0 1

1 2

3 2

3 4

Enter entry point: 0

Enter goal point: 4

Matching Subtree (Path): [0, 1, 2, 3, 4]

6. Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

```
import java.util.*;

class Pair {
    int node, distance;

    Pair(int node, int distance) {
        this.node = node;
        this.distance = distance;
    }
}

public class Shortest_path {

    public static void dijkstra(List<List<Pair>> graph, int start, int end,
int n) {
        int[] dist = new int[n];
        int[] parent = new int[n];

        Arrays.fill(dist, Integer.MAX_VALUE);
        Arrays.fill(parent, -1);

        dist[start] = 0;

        PriorityQueue<Pair> pq = new PriorityQueue<>(Comparator.comparingInt(p
-> p.distance));
        pq.add(new Pair(start, 0));

        while (!pq.isEmpty()) {
            Pair current = pq.poll();

            for (Pair neighbor : graph.get(current.node)) {
                int newDist = current.distance + neighbor.distance;

                if (newDist < dist[neighbor.node]) {
                    dist[neighbor.node] = newDist;
                    parent[neighbor.node] = current.node;
                    pq.add(new Pair(neighbor.node, newDist));
                }
            }
        }

        // Print shortest distance
        System.out.println("Shortest Distance: " + dist[end]);
```

```java
        // Reconstruct path
        List<Integer> path = new ArrayList<>();
        for (int v = end; v != -1; v = parent[v]) {
            path.add(v);
        }
        Collections.reverse(path);

        System.out.println("Shortest Path: " + path);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of nodes: ");
        int n = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        List<List<Pair>> graph = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        System.out.println("Enter edges (u v weight):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            int w = sc.nextInt();

            graph.get(u).add(new Pair(v, w));
            graph.get(v).add(new Pair(u, w)); // undirected
        }

        System.out.print("Enter start node: ");
        int start = sc.nextInt();

        System.out.print("Enter end node: ");
        int end = sc.nextInt();

        dijkstra(graph, start, end, n);

        sc.close();
    }
}
```

OUTPUT :-

Enter number of nodes: 5

Enter number of edges: 6

Enter edges (u v weight):

0 1 2

0 2 4

1 2 1

1 3 7

2 4 3

3 4 1

Enter start node: 0

Enter end node: 4

Shortest Distance: 6

Shortest Path: [0, 1, 2, 4]

7. Write a program to demonstrate the use of all keywords made for exception handling.

```java
public class ExceptionDemo {

    // 1. throws: Declares that this method might throw an exception
    public static void validateAge(int age) throws IllegalArgumentException {

        // 2. assert: Checks a condition (must run with -ea flag)
        assert age >= 0 : "Age cannot be negative!";

        if (age < 18) {
            // 3. throw: Explicitly triggers an exception
            throw new IllegalArgumentException("Underage: Access Denied.");
        } else {
            System.out.println("Access Granted!");
        }
    }

    public static void main(String[] args) {
        // 4. try: Block of code to monitor for errors
        try {
            System.out.println("--- Testing Exception Logic ---");
            validateAge(15);

        } catch (IllegalArgumentException e) {
            // 5. catch: Handles the specific exception thrown
            System.err.println("Caught an Error: " + e.getMessage());

        } finally {
            // 6. finally: Always executes, used for cleanup
            System.out.println("Finally Block: Cleanup logic goes here.");
        }

        System.out.println("\nProgram continues after the catch block...");
    }
}
```

OUTPUT :-

--- Testing Exception Logic ---

Caught an Error: Underage: Access Denied.

Finally Block: Cleanup logic goes here.

Program continues after the catch block...

8. Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

```
import java.util.Scanner;

// Custom Exception
class NoMoneyException extends Exception {
    public NoMoneyException(String message) {
        super(message);
    }
}

// Bank Account Class
class BankAccount {
    double balance;

    // Constructor
    BankAccount(double balance) {
        this.balance = balance;
    }

    // Withdraw Method
    void withdraw(double amount) throws NoMoneyException {
        if (amount > balance) {
            throw new NoMoneyException("Insufficient balance! Cannot
withdraw.");
        } else {
            balance = balance - amount;
            System.out.println("Withdrawal successful!");
            System.out.println("Remaining Balance: " + balance);
        }
    }
}

// Main Class
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter initial balance: ");
        double balance = sc.nextDouble();

        BankAccount account = new BankAccount(balance);

        System.out.print("Enter amount to withdraw: ");
        double amount = sc.nextDouble();
```

```
        try {
            account.withdraw(amount);
        } catch (NoMoneyException e) {
            System.out.println("Exception: " + e.getMessage());
        }

        sc.close();
    }
}
```

OUTPUT :-

Enter initial balance: 30000

Enter amount to withdraw: 25000

Withdrawal successful!

Remaining Balance: 5000.0

9. Write a program which enables to use a platform by a train one at a time.

```java
class Platform {

    // synchronized method (only one thread can access at a time)
    synchronized void usePlatform(String trainName) {
        System.out.println(trainName + " is entering the platform...");

        try {
            Thread.sleep(2000); // simulate time on platform
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        System.out.println(trainName + " is leaving the platform...");
    }
}

// Train class (Thread)
class Train extends Thread {
    Platform platform;
    String trainName;

    Train(Platform platform, String trainName) {
        this.platform = platform;
        this.trainName = trainName;
    }

    public void run() {
        platform.usePlatform(trainName);
    }
}

// Main class
public class Platform_management {
    public static void main(String[] args) {
        Platform platform = new Platform();

        // Multiple trains
        Train t1 = new Train(platform, "Train A");
        Train t2 = new Train(platform, "Train B");
        Train t3 = new Train(platform, "Train C");

        t1.start();
        t2.start();
```

```
        t3.start();
    }
}
```

OUTPUT :-

Train A is entering the platform...

Train A is leaving the platform...

Train B is entering the platform...

Train B is leaving the platform...

Train C is entering the platform...

Train C is leaving the platform...

10. Write a programme of Restaurant and Waiter, which accept order if kitchen if free and sheff deliver orders if he is free.

```java
class Kitchen {
    private boolean orderPlaced = false;
    private String order;

    // Waiter places order
    synchronized void placeOrder(String order) {
        while (orderPlaced) {
            try {
                wait(); // wait if kitchen is busy
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
        this.order = order;
        orderPlaced = true;
        System.out.println("Waiter placed order: " + order);

        notify(); // notify chef
    }

    // Chef prepares and delivers order
    synchronized void prepareOrder() {
        while (!orderPlaced) {
            try {
                wait(); // wait if no order
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
        System.out.println("Chef is preparing: " + order);

        try {
            Thread.sleep(2000); // simulate cooking time
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        System.out.println("Chef delivered: " + order);
        orderPlaced = false;

        notify(); // notify waiter
    }
}

// Waiter Thread
```

```java
class Waiter extends Thread {
    Kitchen kitchen;

    Waiter(Kitchen kitchen) {
        this.kitchen = kitchen;
    }

    public void run() {
        String orders[] = {"Pizza", "Burger", "Pasta"};

        for (String o : orders) {
            kitchen.placeOrder(o);
        }
    }
}

// Chef Thread
class Chef extends Thread {
    Kitchen kitchen;

    Chef(Kitchen kitchen) {
        this.kitchen = kitchen;
    }

    public void run() {
        for (int i = 0; i < 3; i++) {
            kitchen.prepareOrder();
        }
    }
}

// Main Class
public class Hotel_management {
    public static void main(String[] args) {
        Kitchen kitchen = new Kitchen();

        Waiter waiter = new Waiter(kitchen);
        Chef chef = new Chef(kitchen);

        waiter.start();
        chef.start();
    }
}
```

OUTPUT :-

Waiter placed order: Pizza

Chef is preparing: Pizza

Chef delivered: Pizza

Waiter placed order: Burger

Chef is preparing: Burger

Chef delivered: Burger

Waiter placed order: Pasta

Chef is preparing: Pasta

Chef delivered: Pasta